

# Online-updated High-order Collaborative Networks for Single Image Deraining

Cong Wang<sup>1</sup>, Jinshan Pan<sup>2</sup>, Xiao-Ming Wu<sup>1\*</sup>

<sup>1</sup> Department of Computing, The Hong Kong Polytechnic University

<sup>2</sup> School of Computer Science and Engineering, Nanjing University of Science and Technology  
supercong94@gmail.com, sdluran@gmail.com, xiao-ming.wu@polyu.edu.hk

## Abstract

Single image deraining is an important and challenging task for some downstream artificial intelligence applications such as video surveillance and self-driving systems. Most of the existing deep-learning-based methods constrain the network to generate derained images but few of them explore features from intermediate layers, different levels, and different modules which are beneficial for rain streaks removal. In this paper, we propose a high-order collaborative network with multi-scale compact constraints and a bidirectional scale-content similarity mining module to exploit features from deep networks externally and internally for rain streaks removal. Externally, we design a deraining framework with three sub-networks trained in a collaborative manner, where the bottom network transmits intermediate features to the middle network which also receives shallower rainy features from the top network and sends back features to the bottom network. Internally, we enforce multi-scale compact constraints on the intermediate layers of deep networks to learn useful features via a Laplacian pyramid. Further, we develop a bidirectional scale-content similarity mining module to explore features at different scales in a down-to-up and up-to-down manner. To improve the model performance on real-world images, we propose an online-update learning approach, which uses real-world rainy images to fine-tune the network and update the deraining results in a self-supervised manner. Extensive experiments demonstrate that our proposed method performs favorably against eleven state-of-the-art methods on five public synthetic datasets and one real-world dataset. The source code will be available at <https://supercong94.wixsite.com/supercong94>.

## Introduction

Outdoor images taken in rainy conditions have limited visibility, which degrades the performance of various applications such as video surveillance and self-driving systems. Hence, it is essential to recover the degraded images to improve scene visibility and the performance of downstream applications. This paper considers single image deraining.

A rainy image  $O$  can be modeled as a linear combination of a rain-free image  $B$  and a rain streaks image  $R$ :

$$O = B + R. \quad (1)$$

\*Corresponding author

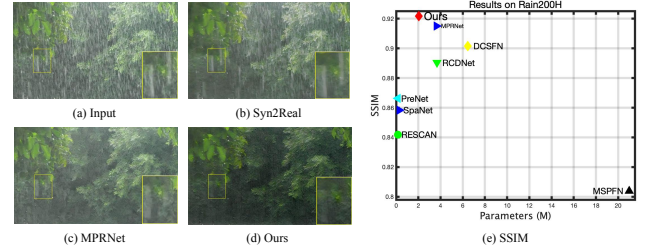


Figure 1: Results on the real-world image (b-d) and the synthetic Rain200H dataset (e). Our proposed method generates a better deraining result on the real-world image and achieves the best performance-parameter trade-off on the synthetic dataset.

Many deraining approaches have been developed based on this simple rainy image model, including prior-based methods and deep-learning-based methods. Prior-based methods usually explore empirical statistical properties of rain streaks and rain-free images, such as image decomposition (?), sparse coding (?), low-rank representation (?), and Gaussian mixture model (?). However, since the priors are based on empirical statistical observations, they do not hold when real-world complex rainy conditions deviate from the simplified assumptions.

Recent years have witnessed the successful application of deep learning methods to image deraining (?????????). Most of these methods develop joint learning networks (???) or explore multi-scale architectures (???) for image deraining. The methods based on joint learning networks include density-guided multi-stream networks (?) and progressive networks motivated by patch similarity and guided by multi-scale architectures (?). While these methods are able to remove rain streaks with the guiding networks, they do not perform well when the guiding process is not accurately estimated. In addition, although multi-scale architectures have been demonstrated to be effective, existing methods usually adopt a straightforward way to fuse features from different scales and do not explore the properties of multi-scale features. Last but not least, existing methods commonly use synthetic datasets to train the deep models due to the lack of well-constructed real-world datasets. However, the gap

between synthetic and real data limits the performance of these methods in real-world applications.

Due to the difficulty of single image deraining, how to fully utilize convolutional features from both shallow and deep layers of a deep model and explore multi-scale features are important for rain streaks removal. In addition, it is essential to develop an effective algorithm to improve the deraining performance on real-world images.

To this end, we propose a high-order collaborative network with multi-scale compact constraints and a bidirectional scale-content mining module to remove rain streaks. The high-order collaborative design allows exploring features from the shallower and deeper layers of different sub-networks collaboratively, while the multi-scale compact constraints are used to effectively learn features from intermediate convolutional layers and the bidirectional scale-content mining module is embedded in an encoder-decoder network to explore features of different scales.

Specifically, the high-order collaborative design contains three sub-networks (bottom, middle, and up) which are learned in a collaborative manner, where the bottom network transmits intermediate features to the middle network which also receives shallower rainy features from the top network and sends back features to the bottom network. The middle and top networks provide deep supervision for the bottom network to better learn and transmit shallow rainy features to higher layers to facilitate deraining. To generate more useful features from a deep network internally, we propose to enforce multi-scale compact constraints on the intermediate layers to learn better features with Laplacian pyramid images. In addition, we develop a bidirectional scale-content similarity mining module in a down-to-up and up-to-down manner to capture long-range dependencies between features at different scales, which is embedded in an encoder-decoder architecture to explore useful features.

Finally, to improve the model performance on real-world images, we propose a simple yet effective online-update learning approach, to fine-tune the model trained on synthetic datasets using real-world data in a self-supervised manner with a KL-Divergence loss function. The proposed network design and online-update learning approach enable our model to achieve state-of-the-art deraining performance, especially on real-world images, as illustrated in Fig. 1.

The main contributions of this paper include:

- We propose a collaborative deraining framework with multi-scale compact constraints to control the learning process in an external and internal manner, with a new bidirectional scale-content similarity mining module to adaptively learn richer feature representations.
- We present a simple yet effective online-update learning approach to fine-tune the model trained on synthetic datasets to adapt to real rainy conditions in a self-supervised manner for real-world image deraining.
- We conduct extensive experiments and ablation studies to evaluate the proposed method. The results demonstrate that our method performs favorably against state-of-the-art methods with fewer parameters on both synthetic and real-world datasets.

## Related Work

In this section, we briefly review recent works on image deraining, which are based on deep learning, as well as some image restoration methods based on similarity mining.

### Single Image Deraining

In recent years, deep-learning-based approaches have dominated the research of image deraining due to the strong representation learning ability of deep neural networks. ? observe that high-frequency details provide more rain streaks details and less background interference and design a deep detail residual network to learn rain streaks. Some deraining methods explore the properties of multi-scale images. ? propose a cross-scale framework to fuse features of different scales from sub-networks. ? design a multi-scale progressive fusion network to transmit and fuse small-scale features to the original scale based on the similarity of multi-scale images. Recurrent networks are also used for deraining. ? develop a recurrent squeeze-and-excitation network with dilation convolution to model channel context relation. ? propose a progressive recurrent network to remove rain streaks stage by stage and analyze the effect of inputs, outputs, and loss functions for image deraining. Some non-local methods are based on attention mechanisms. ? propose to embed a non-local module to an encoder-decoder framework to capture long-range feature dependency for improving representation learning. ? attempt to combine self-attention and scale-aggregation in a self-calibrated network. Besides, some semi-supervised approaches have been proposed for removing rain streaks in real-world images. For example, ? develop a semi-supervised transfer learning approach, and ? design a Gaussian-process-based model that learns on both synthetic and real data.

Different from the above works, our proposed method aims to explore inner structures and useful features of the deraining network in an external and internal manner to train the network for better rain streaks removal. We also propose an effective online-update learning approach to fine-tune the model trained with synthetic data on real data for real-world image deraining.

### Similarity Mining

Similarity mining aims to find the most matched content at the feature level. ? study cross-scale feature mining by exploring the inherent properties of images for single image super-resolution. ? mine the similarity from coarser to finer levels in a multi-scale manner for image restoration. Different from these works that only explore single directional mining (from coarser to finer levels), we in this paper design a bidirectional scale-content similarity mining module to adaptively learn richer features for better deraining.

## Proposed Method

In this section, we introduce each element of the proposed method including high-order collaborative network, multi-scale compact constraint, encoder-decoder structure with a bidirectional scale-content similarity mining module, supervised loss function, and online-update learning.

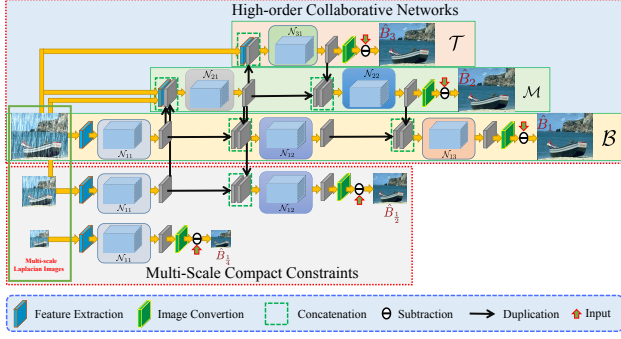


Figure 2: Proposed high-order collaborative networks with multi-scale compact constraints. Each network module  $\mathcal{N}_{mn}$  has a same encoder-decoder structure with the bidirectional scale-content similarity minding module shown in Fig. 3.

### High-order Collaborative Networks

How to exploit features from the intermediate layers and different modules in a collaborative manner is important for image restoration. Here, we design an effective high-order collaborative network<sup>1</sup> to assist each sub-network to learn features in an external manner for better image deraining. The overall network structure of the high-order collaborative network is shown in Fig. 2, which consists of three sub-networks with similar structure. The bottom ( $\mathcal{B}$ ), middle ( $\mathcal{M}$ ), and top ( $\mathcal{T}$ ) sub-networks respectively contain three, two, and one encoder-decoder component with a bidirectional scale-content similarity mining module that will be explained in detail in Sec. .

The three sub-networks can transmit shallower and deeper rainy features to each other in a collaborative manner to improve the performance of the deraining network  $\mathcal{B}$ . Note that  $\mathcal{M}$  and  $\mathcal{T}$  are supervised by ground truth, which can be regarded as a sort of deep supervision in an external manner such that  $\mathcal{M}$  and  $\mathcal{T}$  can transmit useful features to assist  $\mathcal{B}$  to learn better features for deraining. The three sub-networks are trained by:

$$\mathcal{L}_{\text{collaborative}} = \sum_{i=1}^3 \alpha_i (-SSIM(\hat{B}_i, B)), \quad (2)$$

where  $\{\hat{B}_i\}$  ( $i = 1, 2, 3$ ) denote the output of networks  $\mathcal{B}$ ,  $\mathcal{M}$ , and  $\mathcal{T}$  respectively,  $B$  is the ground truth, and  $\alpha_i$  are weight parameters.

### Multi-Scale Compact Constraints

Although the high-order collaborative design enables the bottom sub-network  $\mathcal{B}$  to exploit deeper and shallower features from the other two sub-networks, the features of each intermediate convolutional layer are learned without any constraints, making the solution space too large.

To regularize the solution space of intermediate convolutional layers, we design multi-scale compact constraints

<sup>1</sup>We regard two-stream networks as collaborative networks and three-stream networks as high-order.

(MSCC) in an internal manner to enforce the network modules  $\mathcal{N}_{11}$ ,  $\mathcal{N}_{12}$ , and  $\mathcal{N}_{21}$ , which are parts of the sub-networks  $\mathcal{B}$  and  $\mathcal{M}$ , to learn more useful features to facilitate image deraining. The multi-scale compact constraints are motivated by the multi-scale Laplacian images which can better model image structures than the original scale image. We use them to constrain the intermediate convolutional layers of the network.

We first obtain the Laplacian pyramid images and then use the scaled images to constrain the modules  $\mathcal{N}_{11}$ ,  $\mathcal{N}_{12}$ ,  $\mathcal{N}_{21}$  as shown in Fig. 2. Specifically, the multi-scale compact constraints are enforced by:

$$\mathcal{L}_{\text{mscc}} = \sum_{j=1}^2 \beta_j (-SSIM(\hat{B}_{\frac{1}{2^j}}, B_{\frac{1}{2^j}})), \quad (3)$$

where  $\hat{B}_{\frac{1}{2^j}}$  are the output of intermediate layers as denoted in Fig. 2,  $B_{\frac{1}{2^j}}$  are the corresponding  $\frac{1}{2^j}$  scale Laplacian pyramid rain-free images, and  $\beta_j$  are weight parameters.

### Encoder and Decoder with BiSCSM

Further, we develop a bidirectional scale-content similarity mining module (BiSCSM) to explore similar features from different scales, which is motivated by ???. The architecture is shown in Fig. 3.

The proposed BiSCSM contains Down-to-Up mining and Up-to-Down mining modules. The Up-to-Down mining module is defines as:

$$y_{i,j}^{p \times p} = \frac{1}{\sigma(x,z)} \sum_{g,h} \phi(x_{i,j}^{p \times p}, z_{u,v}^{p \times p}) \theta(x_{g,h}^{p \times p}), \quad (4)$$

where  $\phi(x_{i,j}^{p \times p}, z_{u,v}^{p \times p}) = e^{(W_f x_{i,j}^{p \times p})^T (W_g z_{u,v}^{p \times p})}$ ,  $\theta = W_\theta x_{g,h}^{p \times p}$ ,  $\sigma = \sum_{u,v} \phi(x_{i,j}^{p \times p}, z_{u,v}^{p \times p})$ ,  $y_{i,j}^{p \times p}$  is a  $p \times p$  feature patch located at  $(i, j)$ , and  $W_f, W_g, W_\theta$  are learnable filters. The Down-to-Up mining module is defined similarly as shown in Fig. 4.

Our proposed BiSCSM is different from the one in ??? which only mines patch similarity in a single direction. Our Down-to-Up mining and Up-to-Down mining modules enable capturing bidirectional similar content from large to small scales and from small to large scales to mine rich rainy features. The BiSCSM is embedded in an encoder-decoder framework as in Fig. 3 to learn useful features for rain streaks removal. We also use positional embedding to encode relations among rain streak features. Finally, we fuse the learned features at difference scales.

### Supervised Loss Function

Based on the above network design, the overall loss function for training on synthetic datasets is:

$$\mathcal{L}_{\text{synthetic}} = \mathcal{L}_{\text{collaborative}} + \mathcal{L}_{\text{mscc}}. \quad (5)$$

### Online-update Learning

Since a deraining model trained on synthetic data normally does not generalize well to real-world images, we propose an online-update learning approach to fine-tune the model on real-world rainy images. The key challenges are two-fold. The first is that there are no ground-truth images available

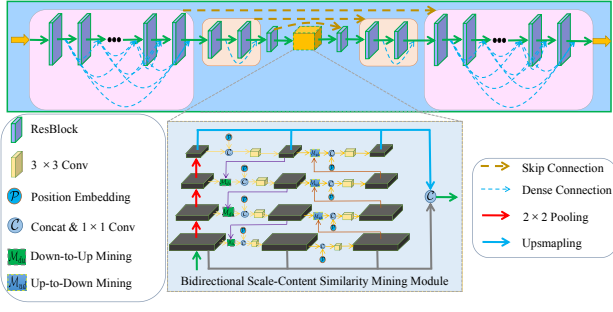


Figure 3: Proposed encoder and decoder with a bidirectional scale-content similarity mining module. The Down-to-Up and Up-to-Down mining modules are shown in Fig. 4.

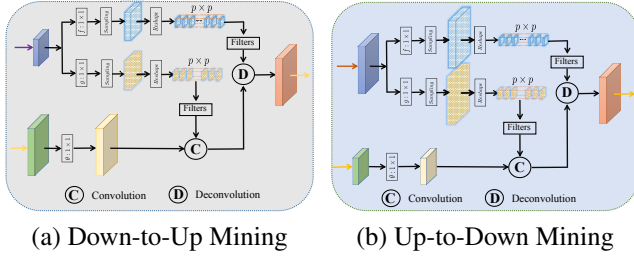


Figure 4: The scale-content similarity mining modules.

for training. The second is how to keep the training stable and improve the model performance. To address these issues, we use the derained results of real-world images generated by the model trained on synthetic data as pseudo ground truth and update the pseudo ground truth at each epoch with the fine-tuned model. The loss function is based on Kullback-Leibler divergence and defined as:

$$\mathcal{L}_{\text{real}} = \underbrace{\|\hat{B}^k - \hat{B}^{k-1}\|_1}_{\text{content term}} + \underbrace{\lambda K L_{\text{Loss}}(\hat{R}^k, \hat{R}_{\text{random}}^{k-1})}_{\text{regularization term}}, \quad (6)$$

where  $\hat{B}^k$  and  $\hat{B}^{k-1}$  are the deraining results of a real-world rainy image  $O$  generated by the model  $\mathcal{B}$  at epoch  $k$  and  $k-1$  respectively. Note that  $\hat{B}^0$  is the initial deraining result produced by  $\mathcal{B}$  after training on synthetic data.  $\hat{R}^k = O - \hat{B}^k$  is the estimated rain streaks of  $O$ .  $\hat{R}_{\text{random}}^{k-1} = O_{\text{random}} - \hat{B}_{\text{random}}^{k-1}$  is the estimated rain streaks of a randomly selected real-world rainy image  $O_{\text{random}}$  and  $\hat{B}_{\text{random}}^{k-1}$  is the deraining result of  $O_{\text{random}}$  at epoch  $k-1$ . The first term of Eq. (6) ensures the content of image background consistent between different epochs and keeps the training stable, and the second term is a regularization term that enforces similarity in rain streaks and improves the deraining performance as the training proceeds (see Fig. 11). The fine-tuning process is illustrated in Fig. 5 and described in Alg. 1.

## Experiments

We compare the proposed approach with 11 state-of-the-art methods (SOTAs), including RESCAN (?), NLEDN (?),

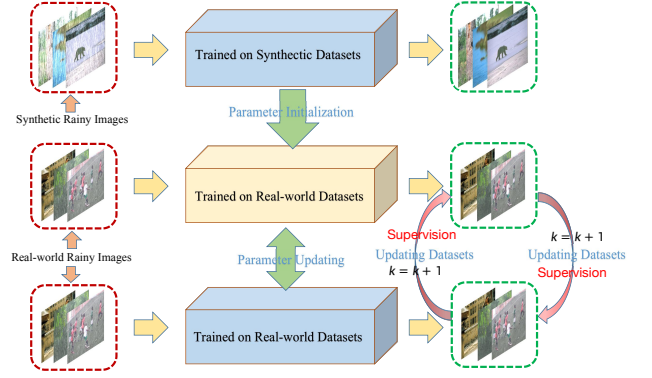


Figure 5: Proposed online-update learning approach.

### Algorithm 1: Online-update Learning on Real-world Data

Preparation: The initial deraining result  $\hat{B}^0$  of a real-world image  $O$  generated by the model trained on synthetic data.

Input:  $\{O, \hat{B}^0\}$ .

Output: Derained image  $\{\hat{B}\}$ .

- 1: While  $k \leq \text{Epoch}_{\text{Real}}$  do:
- 2: Randomly crop training image pairs  $\{O, \hat{B}^{k-1}\}$
- 3: Randomly select a real-world rainy image  $O_{\text{random}}$  and obtain  $\hat{R}_{\text{random}}^{k-1} = O_{\text{random}} - \hat{B}_{\text{random}}^{k-1}$
- 4: Update the deraining model by Eq. (6)
- 5: Output the current deraining result:  $\{\hat{B}^k\}$
- 6: Update the pseudo ground truth:  $\{\hat{B}^{k-1}\} \leftarrow \{\hat{B}^k\}$
- 7:  $k \leftarrow k + 1$
- 8: End while

SSIR (?), PreNet (?), SpaNet (?), DCSFN (?), MSPFN (?), DRDNet (?), RCDNet (?), Syn2Real (?), and MPRNet (?), on five widely used synthetic datasets and a real-world dataset.

## Datasets and Evaluation Criteria

**Synthetic dataset** We use Rain200H (?), Rain200L (?), Rain1200 (?), Rain1400 (?), and Rain12 (?) as the synthetic datasets for training and evaluation. Rain200H is the most challenging dataset, which has 1800 image pairs for training and 200 pairs for testing. Rain200L is the easiest dataset with the same number of training and testing samples as Rain200H. Rain1200 has images with heavy, middle, and light rain, and there are 4000 training images and 400 testing images for each density. Rain1400 has 12600 training samples and 1400 testing samples. Since Rain12 only contains 12 testing samples, we use the model trained on Rain200H to test the deraining results. We use Rain200H as the dataset for ablation study and analysis.

**Real-world dataset** ?, ?, and ? provide a large body of real-world rainy images. We use them to evaluate the deraining results on real-world data.

| Methods            | Rain200H        |                 | Rain200L        |                 | Rain1200        |                 | Rain1400        |                 | Rain12          |                 | # Param |
|--------------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|---------|
|                    | PSNR $\uparrow$ | SSIM $\uparrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ |         |
| RESCAN (ECCV'18)   | 26.661          | 0.8419          | 36.993          | 0.9788          | 32.127          | 0.9028          | 30.969          | 0.9117          | 32.965          | 0.9545          | 0.15M   |
| NLEDN (MM'18)      | 27.315          | 0.8904          | 36.487          | 0.9792          | 32.473          | 0.9198          | 31.014          | 0.9206          | 33.028          | 0.9615          | 1.01M   |
| SSIR (CVPR'19)     | 14.420          | 0.4501          | 23.476          | 0.8026          | 24.427          | 0.7713          | 25.772          | 0.8224          | 24.138          | 0.7768          | 0.06M   |
| PreNet (CVPR'19)   | 27.525          | 0.8663          | 34.266          | 0.9660          | 30.456          | 0.8702          | 30.984          | 0.9156          | 35.095          | 0.9400          | 0.17M   |
| SpaNet (CVPR'19)   | 25.484          | 0.8584          | 36.075          | 0.9774          | 27.099          | 0.8082          | 29.000          | 0.8891          | 33.217          | 0.9546          | 0.28M   |
| DCSFN (MM'20)      | 28.469          | 0.9016          | 37.847          | 0.9842          | 32.275          | 0.9228          | 31.493          | 0.9279          | 35.803          | 0.9683          | 6.45M   |
| MSPFN (CVPR'20)    | 25.553          | 0.8039          | 30.367          | 0.9219          | 30.382          | 0.8860          | 31.514          | 0.9203          | 34.253          | 0.9469          | 21.00M  |
| DRDNet (CVPR'20)   | 15.102          | 0.5028          | 37.465          | 0.9806          | 28.386          | 0.8574          | 28.359          | 0.8574          | 25.435          | 0.7550          | 5.23M   |
| RCDNet (CVPR'20)   | 28.698          | 0.8904          | 38.400          | 0.9841          | 32.273          | 0.9111          | 31.016          | 0.9164          | 31.038          | 0.9069          | 3.67M   |
| Syn2Real (CVPR'20) | 14.495          | 0.4021          | 31.035          | 0.9365          | 28.812          | 0.8400          | 28.582          | 0.8586          | 31.038          | 0.9069          | 2.62M   |
| MPRNet (CVPR'21)   | 29.949          | 0.9151          | 36.610          | 0.9785          | 33.655          | 0.9310          | 32.257          | 0.9325          | 36.578          | 0.9696          | 3.64M   |
| Ours               | <b>29.985</b>   | <b>0.9218</b>   | <b>39.284</b>   | <b>0.9875</b>   | <b>33.718</b>   | <b>0.9327</b>   | <b>32.617</b>   | <b>0.9334</b>   | <b>36.851</b>   | <b>0.9714</b>   | 2.04M   |

Table 1: Quantitative results on five synthetic datasets. The best results are marked in **bold**.  $\uparrow$  denotes higher is better.



Figure 6: Comparison with state-of-the-art methods on synthetic datasets. The proposed network is able to restore better texture.

**Evaluation criteria** We use two widely used metrics, peak signal to noise ratio (PSNR) (?) and structural similarity index measure (SSIM) (?) to evaluate the quality of restored images on synthetic datasets. As there are no ground-truth for real-world rainy images, we only compare the results visually.

### Implementation Details

We set the number of channels of each convolutional layer except the last one as 20, and LeakyReLU with  $\alpha = 0.2$  is used after each convolutional layer except for the last one. For the last layer, we use  $3 \times 3$  convolution without any activation function in  $\mathcal{B}$ ,  $\mathcal{M}$ , and  $\mathcal{T}$ . We randomly crop  $128 \times 128$  image patches as input, and the batch size is set as 12. We use ADAM optimizer (?) to train the network. The initial learning rate is 0.0005, which will be divided by 10 at the 300-th and 400-th epochs, and the model training terminates after 500 epochs. We set  $\lambda = 0.0001$ ,  $\alpha_1 = 1$ ,  $\alpha_2 = 1$ ,  $\alpha_3 = 1$ ,  $\beta_1 = 0.05$ , and  $\beta_2 = 0.001$ . We train the model for 30 epochs on the real-world dataset, i.e.,  $\text{Epoch}_{\text{Real}} = 30$ . Our model is trained with four NVIDIA RTX TITAN GPUs on the Pytorch platform.

### Results and Analysis on Synthetic Datasets

**Comparisons with SOTAs on Synthetic Datasets** Tab. 1 reports the results of our method and SOTAs on five synthetic datasets. We can see that our method achieves the best results on all tested datasets in PSNR and SSIM. We further show some deraining results in Fig. 6. It can be observed that our method can restore better details and textures and obtain clearer background images, while other approaches hand down some rain streaks or lose some details.

**Analysis on BiSCSM** We analyze the effect of different components of BiSCSM and report the results in Tab. 2.

| Experiments          | $M_1$    | $M_2$        | $M_3$        | $M_4$        | $M_5$        | $M_6$    | $M_7$ (Ours)  |
|----------------------|----------|--------------|--------------|--------------|--------------|----------|---------------|
| Sum                  | $\times$ | $\checkmark$ | $\times$     | $\times$     | $\times$     | $\times$ | $\times$      |
| Conv $_{1 \times 1}$ | $\times$ | $\times$     | $\checkmark$ | $\times$     | $\times$     | $\times$ | $\times$      |
| Down-to-Up           | $\times$ | $\times$     | $\times$     | $\checkmark$ | $\times$     | $\times$ | $\times$      |
| Up-to-Down           | $\times$ | $\times$     | $\times$     | $\times$     | $\checkmark$ | $\times$ | $\times$      |
| BiSCSM               | $\times$ | $\times$     | $\times$     | $\times$     | $\times$     | $\times$ | $\checkmark$  |
| Position             | $\times$ | $\checkmark$ | $\checkmark$ | $\checkmark$ | $\checkmark$ | $\times$ | $\checkmark$  |
| PSNR $\uparrow$      | 29.583   | 29.625       | 29.541       | 29.393       | 29.828       | 29.953   | <b>29.985</b> |
| SSIM $\uparrow$      | 0.9168   | 0.9177       | 0.9156       | 0.9172       | 0.9201       | 0.9218   | <b>0.9218</b> |

Table 2: Ablation study of BiSCSM.  $\checkmark$  and  $\times$  denote that the corresponding component is adopted and not adopted, respectively.

Note that the method ( $M_1$ ) does not generate better results if we remove BiSCSM from the encoder-decoder framework. Furthermore, we replace the Down-to-Up and Up-to-Down operations in BiSCSM by element-wise summation ( $M_2$ ) and  $1 \times 1$  convolution ( $M_3$ ) to fuse features at different scales. We find that both operations do not perform well compared to the proposed ones, demonstrating the effectiveness of the mining modules. Fig. 7 visualizes the feature maps before and after BiSCSM. One can see that the features are significantly enhanced after BiSCSM. We also test the single-directional mining modules, i.e., using Down-to-Up ( $M_4$ ) or Up-to-Down ( $M_5$ ) mining module alone, and the results show the bidirectional manner is better and can mine richer features for better deraining ( $M_4$  and  $M_5$  vs.  $M_7$ ). Finally, we observe that position awareness can improve the deraining performance ( $M_6$  vs.  $M_7$ ).

**Analysis on MSCC** Compared with deep models that have no constraints on intermediate layers, our proposed MSCC can make the deep network more compact and learn more useful features. Tab. 3 shows the results of different types of constraints. We observe that the model without any

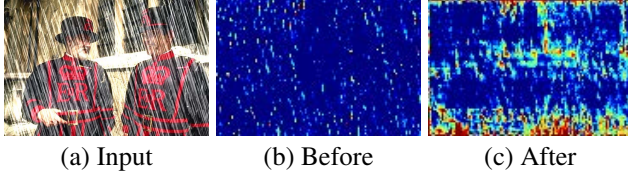


Figure 7: Visualization of the feature maps before and after BiSCSM.

| Experiments     | $R_1$  | $R_2$  | $R_3$  | $R_4$  | $R_5$ (Ours)  |
|-----------------|--------|--------|--------|--------|---------------|
| 1/2 Scale       | ✗      | ✓      | ✗      | ✗      | ✓             |
| 1/4 Scale       | ✗      | ✗      | ✓      | ✗      | ✓             |
| Full Scale      | ✗      | ✗      | ✗      | ✓      | ✗             |
| PSNR $\uparrow$ | 29.824 | 29.879 | 29.839 | 29.970 | <b>29.985</b> |
| SSIM $\uparrow$ | 0.9195 | 0.9209 | 0.9204 | 0.9211 | <b>0.9218</b> |

Table 3: Ablation study of the multi-scale compact constraints.

|                 | w/o $\mathcal{T}$ & w/o $\mathcal{M}$ | w/o $\mathcal{T}$ & w/ $\mathcal{M}$ | Cascaded | w/ $\mathcal{T}$ & w/ $\mathcal{M}$ (Ours) |
|-----------------|---------------------------------------|--------------------------------------|----------|--------------------------------------------|
| PSNR $\uparrow$ | 29.563                                | 29.355                               | 29.642   | <b>29.985</b>                              |
| SSIM $\uparrow$ | 0.9158                                | 0.9146                               | 0.9188   | <b>0.9218</b>                              |

Table 4: Ablation study on collaborative learning.

constraints ( $R_1$ ) performs the worst, while the results get better as we add different constraints and reach the best when both 1/2 ( $R_2$ ) and 1/4 ( $R_3$ ) scale constraints are added.

We also consider replacing the 1/2 and 1/4 scale images with full-scale images to constrain the network. The comparison between  $R_4$  and  $R_5$  demonstrates that full-scale images are less effective than the multi-scale images, which is probably due to the better structures of Laplacian pyramid images.

**Ablation Study on Collaborative Learning** We conduct an ablation study of the proposed collaborative network in Tab. 4. The results show that the model achieves the best performance when the sub-networks  $\mathcal{T}$  and  $\mathcal{M}$  are learned with  $\mathcal{B}$  in a collaborative manner.

We also consider a case by cascading the three sub-networks to form a deep network model. Specifically, we cascade the network modules  $\mathcal{N}_{11}, \mathcal{N}_{12}, \mathcal{N}_{13}, \mathcal{N}_{21}, \mathcal{N}_{22}$ , and  $\mathcal{N}_{31}$  and ensure the cascaded network has roughly the same number of parameters as the collaborative network. We find that the cascaded network does not perform as well as the collaborative network, which further demonstrates the effectiveness of the proposed collaborative learning manner.

**Comparison between Single-stream Learning and Collaborative Learning** In Fig. 8, we provide the comparison of the performance of each sub-network trained in single-stream learning and collaborative learning manners for image deraining. Single-stream learning means training the sub-networks  $\mathcal{T}$ ,  $\mathcal{M}$ , and  $\mathcal{B}$  independently. The results show that the sub-networks trained by collaborative learning perform much better than by single-stream learning. It can be also observed that the sub-network  $\mathcal{T}$  trained by collaborative learning generates comparable results as the sub-

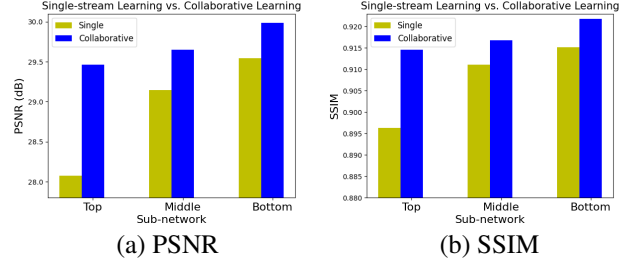


Figure 8: Comparison between single-stream learning and collaborative learning.

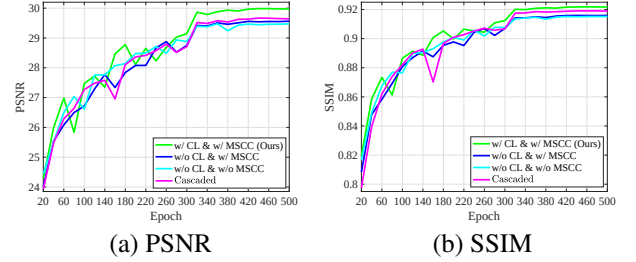


Figure 9: Further ablation study on collaborative learning (CL) and multi-scale compact constraints (MSCC).

network  $\mathcal{B}$  trained by single-stream learning, while the size of  $\mathcal{T}$  is two-thirds of  $\mathcal{B}$ .

**Further Ablation Study on Collaborative Learning and Multi-scale Compact Constraints** Fig. 9 provides a further ablation study of collaborative learning and multi-scale compact constraints, where the learning curves of different variants are plotted. The results show that both collaborative learning and multi-scale compact constraints are useful for improving the deraining performance. It can also be observed that without either collaborative learning or multi-scale compact constraints, the models perform worse than the cascaded network, which further demonstrates the effectiveness of the proposed external and internal learning manners for the deraining task.

## Results and Analysis on Real-world Datasets

**Comparisons with SOTAs on Real-world Images** We further demonstrate the effectiveness of our method on the real-world dataset by comparing with state-of-the-art methods. Fig. 10 presents the deraining results of several challenging cases. It can be observed that our method produces cleaner and clearer deraining results than others, demonstrating its effectiveness in removing rain streaks of real-world rainy images.

### Effectiveness of the Online-update Learning Approach

Fig. 11 presents the ablation study of the online-update learning manner. Compared with only using synthetic data to train the model (Fig. 11(b)) or directly fine-tuning the model on real-world images without updating the pseudo ground

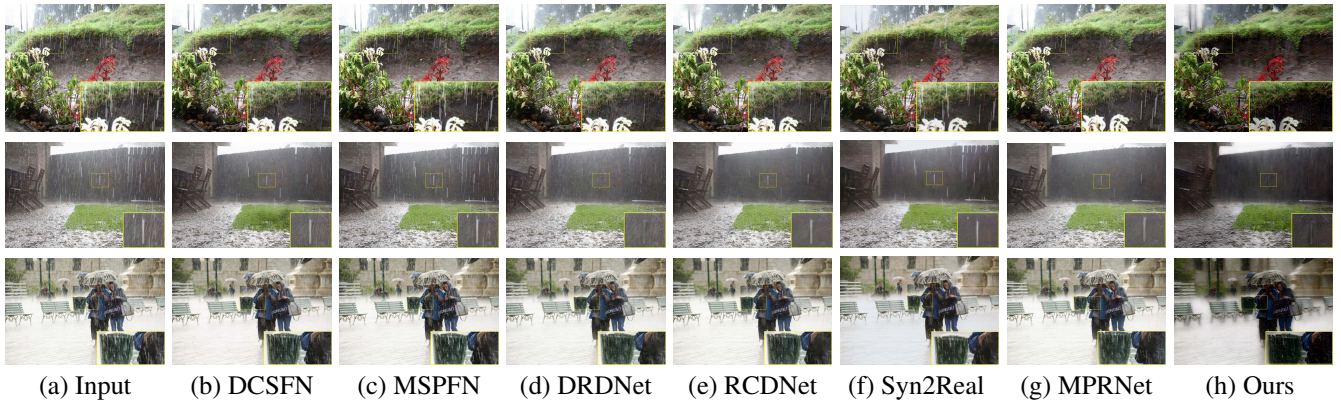


Figure 10: Comparisons with state-of-the-art methods on real-world images. Our proposed online-update learning approach is able to deal with various rainy conditions to better remove rain streaks and even haze and recover clearer images.



Figure 11: Ablation study of the proposed online-update learning approach. (a) Input. (b) Only trained on synthetic data. (c) Directly fine-tuned on real-world images. (d) Online-update learning on real-world images.



Figure 12: Applying the proposed online-update learning approach on DCSFN (?). (a) Input. (b) Only trained on synthetic data. (c) Directly fine-tuned on real-world images. (d) Online-update learning on real-world images.

truth (Fig. 11(c)), our proposed online-update learning manner (Fig. 11(d)) is able to further improve the deraining performance on real-world images, demonstrating its effectiveness.

### Generality of the Online-update Learning Approach

To further demonstrate the effectiveness of the online-update learning approach for real-world image deraining, we apply it on a state-of-the-art method, DCSFN (?). Similar to Fig. 11, Fig. 12 presents the results on a real-world rainy image by different ways of learning. It can be seen that the proposed online-update learning approach can be successfully applied to DCSFN to significantly improve its deraining performance on real-world images. Since the online-update learning approach is generic, we believe it be applied to many other existing methods to improve their performance in real-world image deraining.

## Conclusion

In this paper, we have proposed a high-order collaborative network with multi-scale compact constraints to control the learning process in an external and internal manner for image deraining. We have further developed a bidirectional scale-content similarity mining module to learn useful features at different scales in a down-to-up and up-to-down way to facilitate rain streaks removal. Finally, to improve the deraining performance on real-world images, we have proposed an effective online-update learning approach to fine-tune the deraining model on real-world rainy images in a self-supervised manner. Extensive experiments show that the proposed model outperforms state-of-the-art methods on five public synthetic datasets and one real-world dataset.

## Acknowledgements

We would like to thank the anonymous reviewers for their helpful comments. This work was supported by the Fundamental Research Funds for the Central Universities (No. 30920041109), and the Grant of DaSAIL Project P0030935 funded by PolyU/UGC.

Quibusdam earum perspiciatis harum odit aperiam repellendus, consectetur qui porro error fuga blanditiis velit temporibus labore repudiandae sint, recusandae quam neque fugiat eum cum saepe distinctio, modi dicta voluptatum iusto beatae adipisci, placeat culpa provident?Enim possimus recusandae consectetur architecto voluptatibus explicabo autem, ipsam tenetur nulla sit eius temporibus officia iste rerum asperiores ipsum?Amet incidunt quasi veritatis commodi, ea deserunt voluptatem ut, et optio hic rem commodi veritatis harum consequuntur ex voluptatem, sed id porro doloremque eaque consectetur dolorem provident assumenda?Quam reiciendis sit officiis possimus, incidunt quis animi fugiat ut neque illum veniam?Expedita dolorum maiores quod perspiciatis, officia similique impedit, reiciendis molestias ipsam.Maxime veniam corporis, veritatis deserunt facere?Repellat inventore quasi assumenda magni nemo officia modi, expedita corporis maiores quibusdam optio alias voluptate, reprehenderit quis esse neque excepturi

adipisci laborum odit at, odio ducimus dolorum praesentium  
sunt ipsa sit, libero veniam labore autem sint ex?Ut eaque  
modi ipsam dolore error aliquam impedit